

Backend Development

Chapter 1: C# Programming languages

C# Fundamentals

What is C# ?

- C# (C-Sharp) is a programming language developed by Microsoft that runs on the .NET Framework.
- C# has roots from the C family, and the language is close to other popular languages like C++ and Java.
- The first version was released in year 2002. The latest version, **C# 12**, was released in November 2023.
- C# is used to develop web applications, desktop applications, mobile apps, games and much more.
- A modern, object-oriented programming language developed by Microsoft.
- Part of the .NET ecosystem.
- Combines the best of C++ and Java.

2. INTEGRATED DEVELOPMENT ENVIRONMENT (IDE)



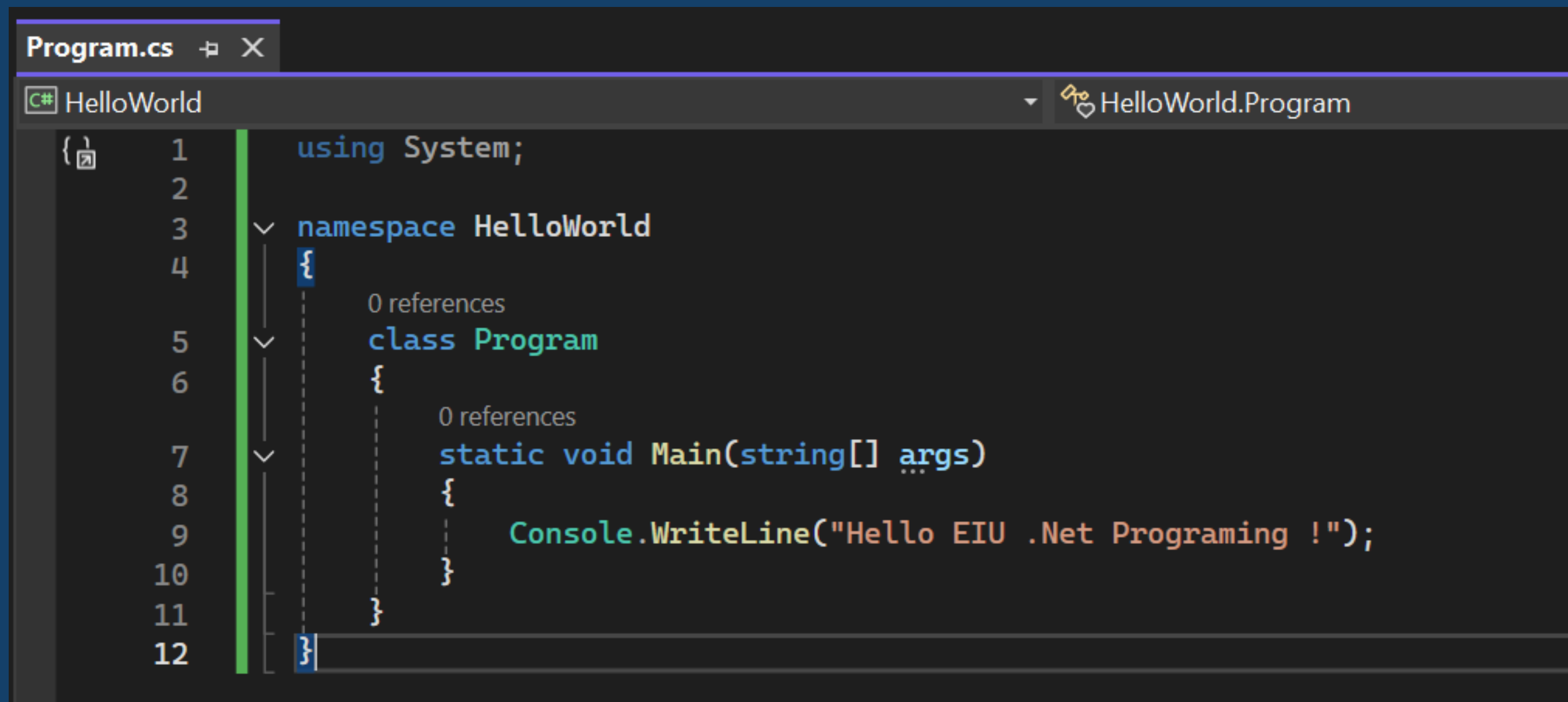
Visual Studio Installer

Visual Studio Community

A free, fully-featured, and extensible IDE for creating modern applications for Windows, Android, and iOS, as well as web applications and cloud services.



The Basic C# file when create a console project that called "Program.cs"



```
Program.cs
C# HelloWorld HelloWorld.Program
1 using System;
2
3 namespace HelloWorld
4 {
5     0 references
6     class Program
7     {
8         0 references
9         static void Main(string[] args)
10        {
11            Console.WriteLine("Hello EIU .Net Programing !");
12        }
13    }
```

Example explained

- **Line 1:** using System means that we can use classes from the System namespace.
- **Line 2:** A blank line. C# ignores white space. However, multiple lines makes the code more readable.
- **Line 3:** namespace is used to organize your code, and it is a container for classes and other namespaces.
- **Line 4:** The curly braces {} marks the beginning and the end of a block of code.
- **Line 5:** class is a container for data and methods, which brings functionality to your program. Every line of code that runs in C# must be inside a class. In our example, we named the class Program.
- **Line 7:** Another thing that always appear in a C# program is the Main method. Any code inside its curly brackets {} will be executed. You don't have to understand the keywords before and after Main. You will get to know them bit by bit while reading this tutorial.
- **Line 9:** Console is a class of the System namespace, which has a WriteLine() method that is used to output/print text. In our example, it will output "Hello World!".
If you omit the using System line, you would have to write System.Console.WriteLine() to print/output text.

How can we comment code in the C# programming language ?

```
/* Comment
 * for
 * multiple
 * line */

// Comment for single line
```

```
17
18
19  ✓
20
21  *
22  * Console.WriteLine("Hello EIU .Net Programing !"); */
23
24  // This is the comment for single line
25  //Console.WriteLine("Hello EIU .Net Programing !");
26  }
27  }
28  }
```

- int** - stores integers (whole numbers), without decimals, such as 123 or -123
- double** - stores floating point numbers, with decimals, such as 19.99 or -19.99
- char** - stores single characters, such as 'a' or 'B'. Char values are surrounded by single quotes
- string** - stores text, such as "Hello World". String values are surrounded by double quotes
- bool** - stores values with two states: true or false

Syntax

```
type variableName = value;
```

```
string subjectName = "EIU .Net Programming";  
int myNum = 99;
```

Constants

- Cannot declare a constant variable without assigning the value
- Read-only
- Cannot overwrite existing values

```
21  
22  
23  
24  
const int myNum = 2024;  
myNum = 2023; // error
```

How to use variables

```
string name = "Taylor Swift";  
Console.WriteLine("Hello " + name);
```

```
string firstName = ".NET ";  
string lastName = "Programming";  
string fullName = firstName + lastName;  
Console.WriteLine(fullName);
```

```
int x = 99;  
int y = 1;  
Console.WriteLine(x + y); // Print the value of x + y
```

Identifiers and clean variables

```
// Good  
bool isDeleted = false;  
  
// OK, but not so easy to understand what d actually is  
bool d = false;
```

Data Type	Size	Description
int	4 bytes	Stores whole numbers from -2,147,483,648 to 2,147,483,647
long	8 bytes	Stores whole numbers from -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
float	4 bytes	Stores fractional numbers. Sufficient for storing 6 to 7 decimal digits
double	8 bytes	Stores fractional numbers. Sufficient for storing 15 decimal digits
bool	1 bit	Stores true or false values
char	2 bytes	Stores a single character/letter, surrounded by single quotes
string	2 bytes per character	Stores a sequence of characters, surrounded by double quotes

```

int myNum = 5;           // Integer (whole number)
double myDoubleNum = 5.99D; // Floating point number
char myLetter = 'D';      // Character
bool myBool = true;       // Boolean
string myText = "Hello";  // String
  
```

Type casting is when you assign a value of one data type to another type.

Implicit Casting (automatically) - converting a smaller type to a larger type size
char -> int -> long -> float -> double

Explicit Casting (manually) - converting a larger type to a smaller size type
double -> float -> long -> int -> char

```
//Implicit Casting
int myInt = 9;
double myDouble = myInt;      // Automatic casting: int to double

Console.WriteLine(myInt);     // Outputs 9
Console.WriteLine(myDouble);  // Outputs 9
```

```
//Explicit Casting
double myDouble = 9.78;
int myInt = (int)myDouble;    // Manual casting: double to int

Console.WriteLine(myDouble);  // Outputs 9.78
Console.WriteLine(myInt);     // Outputs 9
```

```
// Type your username and press enter
Console.WriteLine("Enter username:");

// Create a string variable and get user input from the keyboard and store it in the variable
string userName = Console.ReadLine();

// Print the value of the variable (userName), which will display the input value
Console.WriteLine("Username is: " + userName);
```

Handle error for user Input

```
Console.WriteLine("Enter your age:");  
int age = Console.ReadLine();  
Console.WriteLine("Your age is: " + age);
```

	Code	Description
✖	CS0029	Cannot implicitly convert type 'string' to 'int'

```
Console.WriteLine("Enter your age:");  
int age = Convert.ToInt32(Console.ReadLine());  
Console.WriteLine("Your age is: " + age);
```

Microsoft Visual Studio Debug Console

```
Enter your age:  
18  
Your age is: 18
```

Operators are used to perform operations on variables and values.

Operator	Name	Description	Example
+	Addition	Adds together two values	$x + y$
-	Subtraction	Subtracts one value from another	$x - y$
*	Multiplication	Multiplies two values	$x * y$
/	Division	Divides one value by another	x / y
%	Modulus	Returns the division remainder	$x \% y$
++	Increment	Increases the value of a variable by 1	$x++$
--	Decrement	Decreases the value of a variable by 1	$x--$

Assignment Operators

- Assignment operators are used to assign values to variables.

Operator	Example	Same As
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
&=	x &= 3	x = x & 3
=	x = 3	x = x 3
^=	x ^= 3	x = x ^ 3
>>=	x >>= 3	x = x >> 3
<<=	x <<= 3	x = x << 3

Comparison Operators

- Comparison operators are used to compare two values (or variables). This is important in programming, because it helps us to find answers and make decisions.

Operator	Name	Example
==	Equal to	<code>x == y</code>
!=	Not equal	<code>x != y</code>
>	Greater than	<code>x > y</code>
<	Less than	<code>x < y</code>
>=	Greater than or equal to	<code>x >= y</code>
<=	Less than or equal to	<code>x <= y</code>

```
int x = 5;
int y = 3;
Console.WriteLine(x > y); // returns True because 5 is greater than 3
```

Microsoft Visual Studio Debug Console

True

Logical Operators

- Logical operators are used to determine the logic between variables or values

Operator	Name	Description	Example
&&	Logical and	Returns True if both statements are true	<code>x < 5 && x < 10</code>
	Logical or	Returns True if one of the statements is true	<code>x < 5 x < 4</code>
!	Logical not	Reverse the result, returns False if the result is true	<code>!(x < 5 && x < 10)</code>

A string variable contains a collection of characters surrounded by double quotes

A string in C# is actually an object, which contain properties and methods that can perform certain operations on strings.

Length:

```
string txt = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";  
Console.WriteLine("The length of the txt string is: " + txt.Length);
```

Microsoft Visual Studio Debug Console
The length of the txt string is: 26

Uppercase or lowercase:

```
string txt = "Hello World";  
Console.WriteLine(txt.ToUpper()); // Outputs "HELLO WORLD"  
Console.WriteLine(txt.ToLower()); // Outputs "hello world"
```

Microsoft Visual Studio Debug Console
HELLO WORLD
hello world

String Concatenation

The “+” operator can be used between strings to combine them. This is called concatenation:

```
string firstName = ".NET ";  
string lastName = "Programming";  
string name = firstName + lastName;  
Console.WriteLine(name);
```



Microsoft Visual Studio Debug Console



.NET Programming

String Interpolation

```
string firstName = ".NET ";  
string lastName = "Programming";  
string name = $"My course is: {firstName} {lastName}";  
Console.WriteLine(name);
```



Microsoft Visual Studio Debug Console



My course is: .NET Programming

Special Characters

The strings must be written within quotes, C# will misunderstand this string, and generate an error:

```
string txt = "I'm learning the ".NET Programming" course.";
```

Escape character	Result	Description
\'	'	Single quote
\"	"	Double quote
\\	\	Backslash

```
string txt = "I'm learning the \".NET Programming\" course.";
Console.WriteLine(txt);
```

Microsoft Visual Studio Debug Console

I'm learning the ".NET Programming" course.

C# has a `bool` data type, which can take the values `true` or `false`.

```
bool isDotNetFun = true;
bool isFishTasty = false;
Console.WriteLine(isDotNetFun); // Outputs True
Console.WriteLine(isFishTasty); // Outputs False
```



Microsoft Visual Studio Debug Console



```
True
False
```

Boolean Expression

- A Boolean expression returns a boolean value: True or False, by comparing values/variables.
- Useful to build logic, and find answers.

```
int x = 10;  
int y = 9;  
Console.WriteLine(x > y); // returns True, because 10 is higher than 9
```

Microsoft Visual Studio Debug

True

Boolean Expression

More Example:

```
int x = 10;
Console.WriteLine(x == 10); // returns True, because the value of x is equal to 10
Console.WriteLine(10 == 15); // returns False, because 10 is not equal to 15
int myAge = 25;
int votingAge = 18;
if (myAge >= votingAge)
{
    Console.WriteLine("Old enough to vote!");
}
else
{
    Console.WriteLine("Not old enough to vote.");
}
```

Conditions

C# supports the usual logical conditions from mathematics:

- Less than: $a < b$
- Less than or equal to: $a \leq b$
- Greater than: $a > b$
- Greater than or equal to: $a \geq b$
- Equal to $a == b$
- Not Equal to: $a != b$

C# has the following conditional statements:

- Use if to specify a block of code to be executed, if a specified condition is true
- Use else to specify a block of code to be executed, if the same condition is false
- Use else if to specify a new condition to test, if the first condition is false
- Use switch to specify many alternative blocks of code to be executed

The If Statements

```
if (condition)
{
    // block of code to be executed if the condition is True
}

if (20 > 18)
{
    Console.WriteLine("20 is greater than 18");
}

int x = 20;
int y = 18;
if (x > y)
{
    Console.WriteLine("x is greater than y");
}
```

The Else Statement

```
if (condition)
{
    // block of code to be executed if the condition is True
}
else
{
    // block of code to be executed if the condition is False
}

int time = 20;
if (time < 18)
{
    Console.WriteLine("Good day.");
}
else
{
    Console.WriteLine("Good evening.");
}
// Outputs "Good evening."
```

The Else If Statement

```
if (condition1)
{
    // block of code to be executed if condition1 is True
}
else if (condition1)
{
    // block of code to be executed if the condition1 is false and condition2 is True
}
else
{
    // block of code to be executed if the condition1 is false and condition2 is False
}

int time = 22;
if (time < 10)
{
    Console.WriteLine("Good morning.");
}
else if (time < 20)
{
    Console.WriteLine("Good day.");
}
else
{
    Console.WriteLine("Good evening.");
}
// Outputs "Good evening."
```

Short Hand If...Else

variable = (condition) ? expressionTrue : expressionFalse;

```
int time = 20;  
string result = (time < 18) ? "Good day." : "Good evening.";  
Console.WriteLine(result);
```



Microsoft Visual Studio Debug Console



Good evening.

Use the switch statement to select one of many code blocks to be executed.

```
switch(expression)
{
    case 1:
        // code block
        break;
    case 2:
        // code block
        break;
    default:
        // code block
        break;
}
```

Example:

```
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48

int day = 4;
switch (day)
{
    case 1:
        Console.WriteLine("Monday");
        break;
    case 2:
        Console.WriteLine("Tuesday");
        break;
    case 3:
        Console.WriteLine("Wednesday");
        break;
    case 4:
        Console.WriteLine("Thursday");
        break;
    case 5:
        Console.WriteLine("Friday");
        break;
    case 6:
        Console.WriteLine("Saturday");
        break;
    case 7:
        Console.WriteLine("Sunday");
        break;
}
// Outputs "Thursday" (day 4)
```

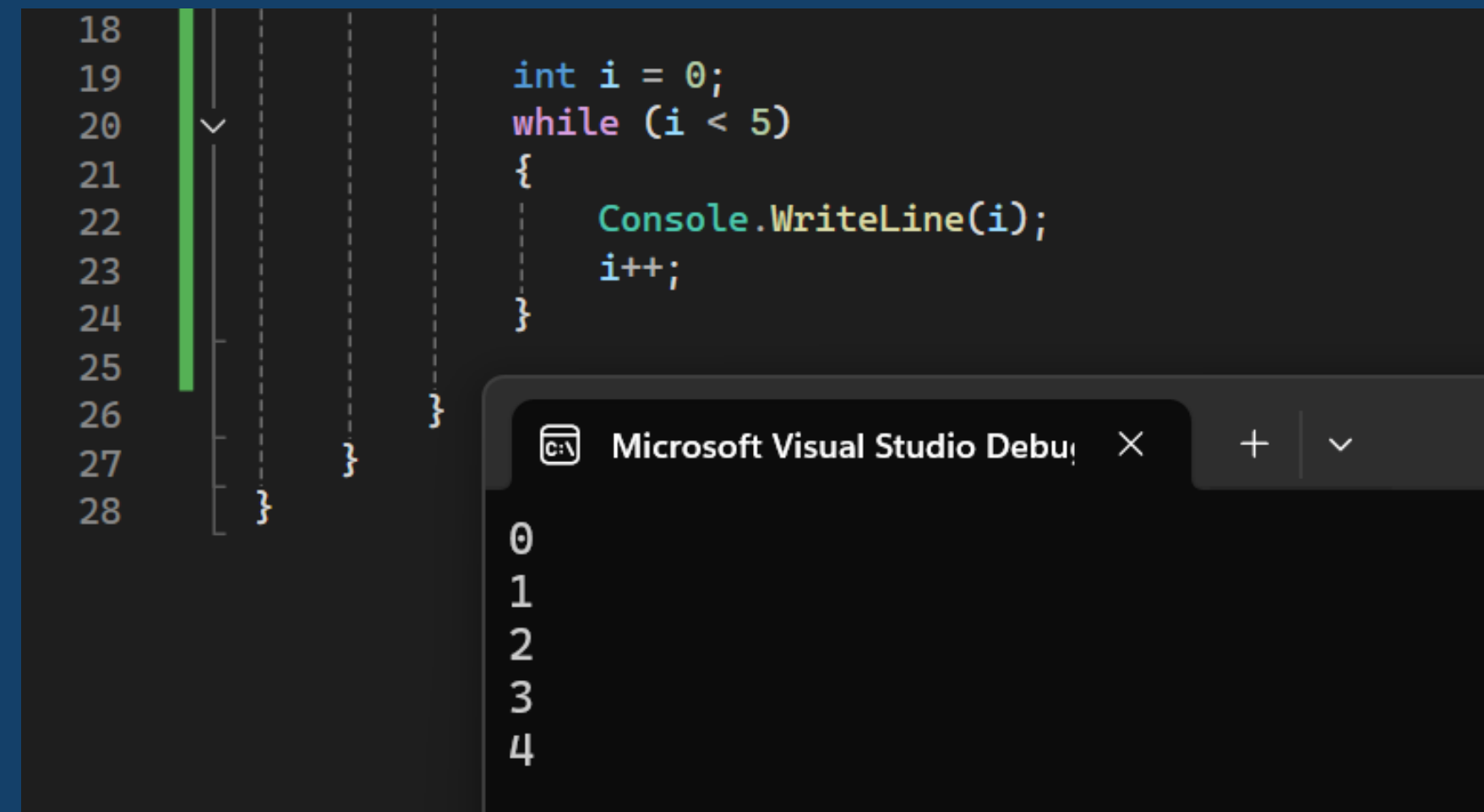
Microsoft Visual Studio Debug Console

Thursday

While Loop

Syntax:

```
while (condition)
{
    // code block to be executed
}
```



The screenshot shows a code editor with a C# program. The code defines an integer variable `i` and enters a `while` loop that continues as long as `i` is less than 5. Inside the loop, the current value of `i` is printed to the console, and `i` is incremented by 1. The output window on the right shows the sequence of numbers 0 through 4, which are the values of `i` during each iteration of the loop.

```
18  
19  
20  
21  
22  
23  
24  
25  
26  
27  
28
```

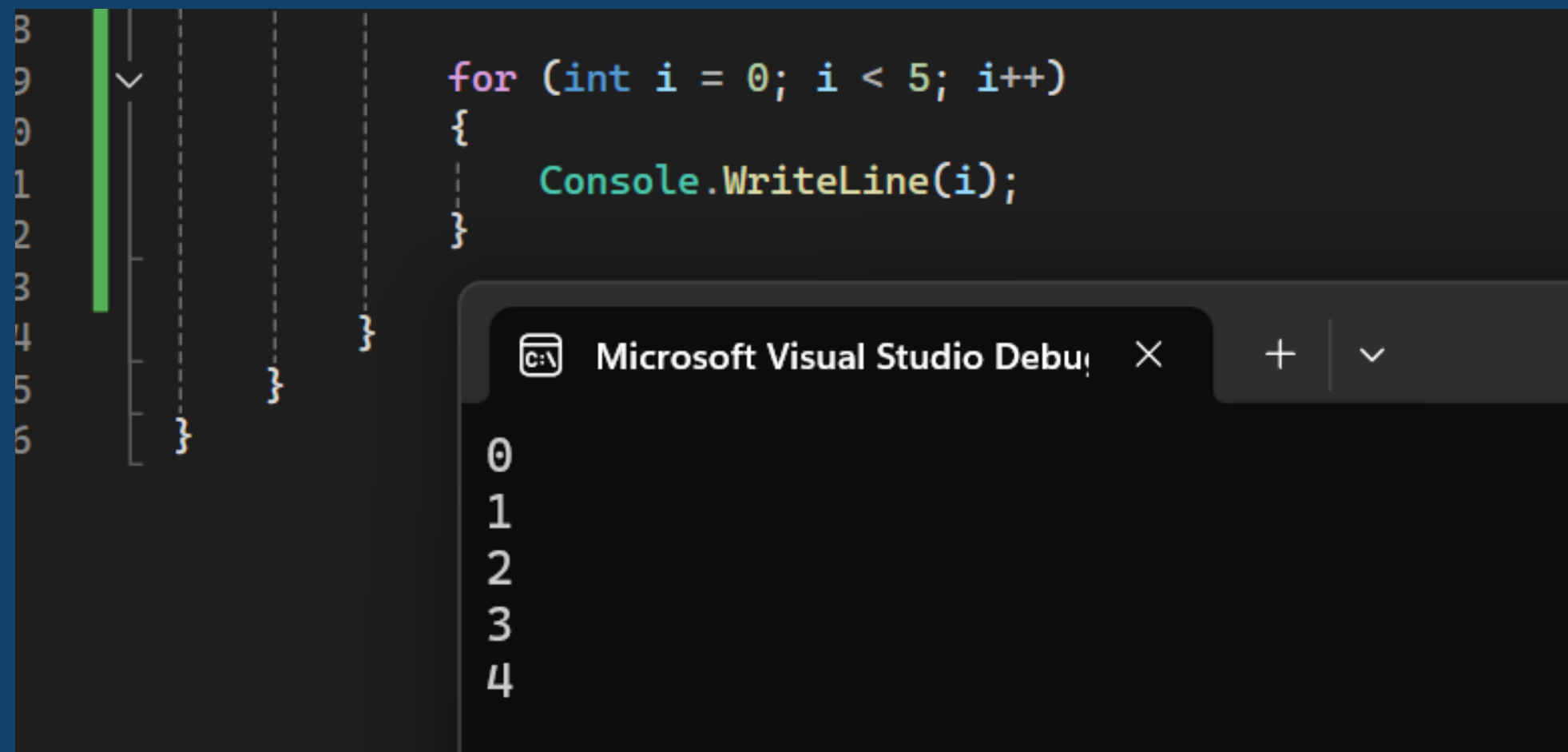
```
int i = 0;  
while (i < 5)  
{  
    Console.WriteLine(i);  
    i++;  
}
```

Microsoft Visual Studio Debug Console

0
1
2
3
4

For Loop Syntax:

```
for (statement 1; statement 2; statement 3)
{
    // code block to be executed
}
```



The screenshot shows a C# program in Visual Studio. The code defines a for loop that iterates from i = 0 to i = 4, printing each value to the console. The output window shows the sequence of numbers 0, 1, 2, 3, and 4.

```
for (int i = 0; i < 5; i++)
{
    Console.WriteLine(i);
}
```

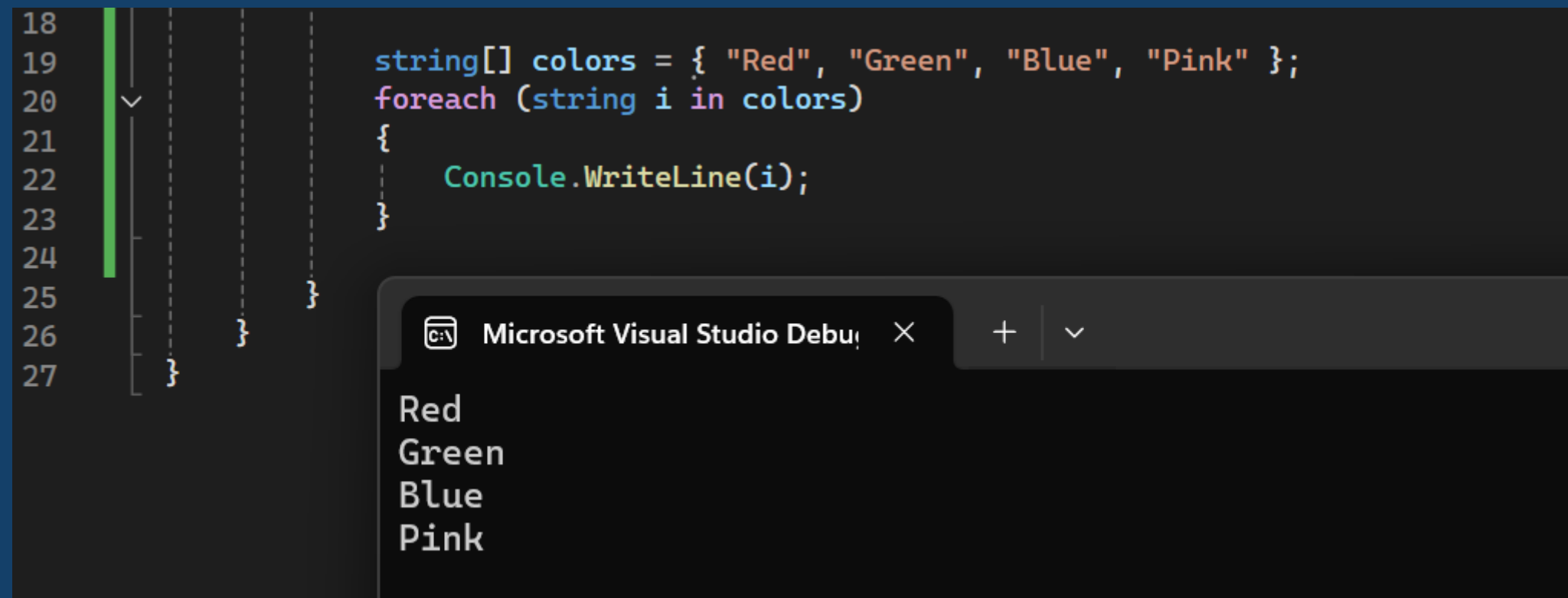
Microsoft Visual Studio Debug Console Output:

```
0
1
2
3
4
```

Foreach Loop

Syntax:

```
foreach (type variableName in arrayName)
{
    // code block to be executed
}
```



The screenshot shows a code editor with the following C# code:

```
18
19
20 string[] colors = { "Red", "Green", "Blue", "Pink" };
21 foreach (string i in colors)
22 {
23     Console.WriteLine(i);
24 }
25
26
27
```

Below the code editor, a console window titled "Microsoft Visual Studio Debug" displays the output of the program:

```
Red
Green
Blue
Pink
```

Break and Continue

- **BREAK:** The break statement can also be used to jump out of a loop.
- **CONTINUE:** The continue statement breaks one iteration (in the loop), if a specified condition occurs, and continues with the next iteration in the loop.

```
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
```

```
for (int i = 0; i < 10; i++)
{
    if (i == 4)
    {
        break;
    }
    Console.WriteLine(i);
}
```

Microsoft Visual Studio Debug Console

0
1
2
3

```
19
20
21
22
23
24
25
26
27
28
29
30
31
```

```
for (int i = 0; i < 10; i++)
{
    if (i == 4)
    {
        continue;
    }
    Console.WriteLine(i);
}
```

Microsoft Visual Studio Debug Console

0
1
2
3
5
6
7
8
9

Arrays are used to store multiple values in a single variable, instead of declaring separate variables for each value

```
string[] carsList = { "Volvo", "BMW", "Ford", "Mazda" };  
int[] myNum = { 10, 20, 30, 40 };
```

Access the Elements of an Array:

```
string[] cars = { "Volvo", "BMW", "Ford", "Mazda" };  
Console.WriteLine(cars[0]);  
// Outputs Volvo
```

Change an Array Element:

```
cars[0] = "Huyndai";
```

Array Length:

```
string[] cars = { "Volvo", "BMW", "Ford", "Mazda" };  
Console.WriteLine(cars.Length);  
// Outputs 4
```

Other Ways to Create an Array

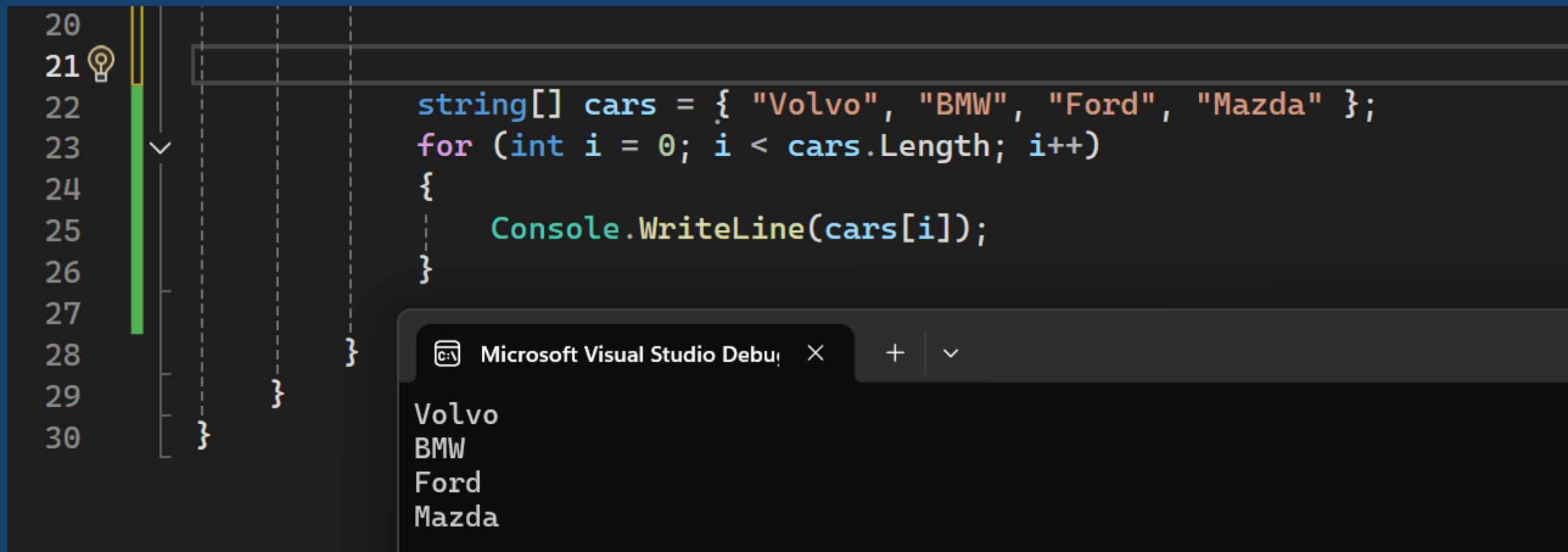
```
// Create an array of four elements, and add values later
string[] carsClassA = new string[4];

// Create an array of four elements and add values right away
string[] carsClassB = new string[4] { "Volvo", "BMW", "Ford", "Mazda" };

// Create an array of four elements without specifying the size
string[] carsClassC = new string[] { "Volvo", "BMW", "Ford", "Mazda" };

// Create an array of four elements, omitting the new keyword, and without specifying the size
string[] carsClassD = { "Volvo", "BMW", "Ford", "Mazda" };
```

Loop Through Arrays



The screenshot shows a code editor with a C# program. The code defines an array of car names and iterates through it using a for loop. The console output shows the names of the cars: Volvo, BMW, Ford, and Mazda.

```
20  
21  
22 string[] cars = { "Volvo", "BMW", "Ford", "Mazda" };  
23 for (int i = 0; i < cars.Length; i++)  
24 {  
25     Console.WriteLine(cars[i]);  
26 }  
27  
28 }  
29  
30 }
```

Microsoft Visual Studio Debug Console

Volvo
BMW
Ford
Mazda

```
class Program
{
    static void MyMethod()
    {
        // code to be executed
    }
}
```

MyMethod(): is the name of the method

static means that the method belongs to the Program class and not an object of the Program class.

void means that this method does not have a return value. You can choose another type like int, string, array, object,....

Call a Method

```
using System;
using HelloWorldx;

namespace HelloWorld
{
    0 references
    class Program
    {
        1 reference
        static void MyMethod()
        {
            Console.WriteLine("I just got executed!");
        }

        0 references
        static void Main(string[] args)
        {
            MyMethod();
        }
    }
}
```

Method Parameters

```
0 references
class Program
{
    3 references
    static void MyMethod(string fname)
    {
        Console.WriteLine(fname + " is a CIT's member");
    }

    0 references
    static void Main(string[] args)
    {
        MyMethod("Cuong");
        MyMethod("Xuan");
        MyMethod("Thuy");
    }
}
```

Microsoft Visual Studio Debug Console

```
Cuong is a CIT's member
Xuan is a CIT's member
Thuy is a CIT's member
```

Default Parameter Value

```
1  using System;
2  using HelloWorld;
3
4  namespace HelloWorld
5  {
6      0 references
7      class Program
8      {
9          4 references
10         static void MyMethod(string fname = "Method Have Default Parameter Value")
11         {
12             Console.WriteLine(fname + " is a CIT's member");
13         }
14
15         0 references
16         static void Main(string[] args)
17         {
18             MyMethod();
19             MyMethod("Cuong");
20             MyMethod("Xuan");
21             MyMethod("Thuy");
22         }
23     }
24
25 
```

Microsoft Visual Studio Debug Console

```
Method Have Default Parameter Value is a CIT's member
Cuong is a CIT's member
Xuan is a CIT's member
Thuy is a CIT's member
```

Return Values

```
1 reference
static int MyMethod(int x)
{
    return 5 + x;
}

0 references
static void Main(string[] args)
{
    Console.WriteLine(MyMethod(3));
}

// Outputs 8 (5 + 3)
```



Microsoft Visual Studio Debug Console



8

Named Arguments

```
1 reference
static void MyMethod(string child1, string child2, string child3)
{
    Console.WriteLine("The youngest child is: " + child3);
}

0 references
static void Main(string[] args)
{
    MyMethod(child3: "John", child1: "Liam", child2: "Liam");
}
```



Microsoft Visual Studio Debug



The youngest child is: John

Object-oriented programming with C#

- OOP stands for Object-Oriented Programming.
- Object-oriented programming has several advantages over procedural programming:
 - OOP is faster and easier to execute
 - OOP provides a clear structure for the programs
 - OOP makes the code easier to maintain, modify and debug
 - OOP makes it possible to create full reusable applications with less code and shorter development time
- Classes and objects are the two main aspects of object-oriented programming.
- A class is a template for objects, and an object is an instance of a class

To create a class, use the class keyword:

```
// Create a class named "Car" with a variable color:  
0 references  
class Car  
{  
    string color = "red";  
}
```

Create an Object: Create an object called "myObj" and use it to print the value of color:

```
class Car  
{  
    string color = "red";  
  
    0 references  
    static void Main(string[] args)  
    {  
        Car myObj = new Car();  
        Console.WriteLine(myObj.color);  
    }  
}
```

Microsoft Visual Studio Debug

red

Fields and methods inside classes are often referred to as "Class Members":

```
6  class MyClass
7  {
8      // Class members
9      string color = "red";           // field
10     int maxSpeed = 200;              // field
11                                     0 references
12     public void fullThrottle()      // method
13     {
14         Console.WriteLine("The car is going as fast as it can!");
15     }
16 }
```

Fields

Variables inside a class are called fields, you can access them by creating an object of the class, and by using the dot syntax (.).

The following example will create an object of the Car class, with the name myObj. Then we print the value of the fields color and maxSpeed:

```
17 class Car
18 {
19     string color = "red";
20     int maxSpeed = 200;
21
22     0 references
23     static void Main(string[] args)
24     {
25         Car myObj = new Car();
26         Console.WriteLine(myObj.color);
27         Console.WriteLine(myObj.maxSpeed);
28     }
29 }
```

Object Methods

Methods normally belong to a class, and they define how an object of a class behaves.

Just like with fields, you can access methods with the dot syntax. However, note that the method must be public. And remember that we use the name of the method followed by two parentheses () and a semicolon ; to call (execute) the method:

```
2 references
class Car
{
    string color;           // field
    int maxSpeed;           // field
    1 reference
    public void fullThrottle() // method
    {
        Console.WriteLine("The car is going as fast as it can!");
    }

    0 references
    static void Main(string[] args)
    {
        Car myObj = new Car();
        myObj.fullThrottle(); // Call the method
    }
}
```

A constructor is a special method that is used to initialize objects. The advantage of a constructor, is that it is called when an object of a class is created. It can be used to set initial values for fields:

```
// Create a Car class
3 references
class Car
{
    public string model; // Create a field

    // Create a class constructor for the Car class
    1 reference
    public Car()
    {
        model = "Mustang"; // Set the initial value for model
    }

    0 references
    static void Main(string[] args)
    {
        Car Ford = new Car(); // Create an object of the Car Class (this will call the constructor)
        Console.WriteLine(Ford.model); // Print the value of model
    }
}

// Outputs "Mustang"
```

Constructor Parameters

Constructors can also take parameters, which is used to initialize fields.

The following example adds a **string modelName** parameter to the constructor. Inside the constructor we set **model** to **modelName** (**model=modelName**). When we call the constructor, we pass a parameter to the constructor ("**Mustang**"), which will set the value of **model** to "**Mustang**":

```
5 3 references
6 class Car
7 {
8     public string model;
9
10 // Create a class constructor with a parameter
11 1 reference
12 public Car(string modelName)
13 {
14     model = modelName;
15 }
16
17 0 references
18 static void Main(string[] args)
19 {
20     Car Ford = new Car("Honda");
21     Console.WriteLine(Ford.model);
22 }
23
24 // Outputs "Honda"
25
26
27
28
```

Microsoft Visual Studio Debug Console: Honda

public string color;

The **public** keyword is an access modifier, which is used to set the access level/visibility for classes, fields, methods and properties.

C# has the following access modifiers:

Modifier	Description
public	The code is accessible for all classes
private	The code is only accessible within the same class
protected	The code is accessible within the same class, or in a class that is inherited from that class.
internal	The code is only accessible within its own assembly, but not from another assembly.

Private Modifier

If you declare a field with a **private** access modifier, it can only be accessed within the same class.

If you try to access it outside the class, an error will occur.

```

5 | 2 references
6 | class Car
7 | {
8 |     private string model = "Mustang";
9 |
10 | 0 references
11 | static void Main(string[] args)
12 | {
13 |     Car myObj = new Car();
14 |     Console.WriteLine(myObj.model);
15 | }
16 |
17 | // output is Mustang
18 |
19 |
20 |

```

Microsoft Visual Studio Debug Console

Mustang

```

5 | 2 references
6 | class Car
7 | {
8 |     private string model = "Mustang";
9 | }
10 |
11 | 0 references
12 | class Program
13 | {
14 |     0 references
15 | static void Main(string[] args)
16 | {
17 |     Car myObj = new Car();
18 |     Console.WriteLine(myObj.model);
19 | }
20 |

```

	Code	Description
×	CS0122	'Car.model' is inaccessible due to its protection level

WHY ACCESS MODIFIERS?

To control the visibility of class members (the security level of each individual class and class member).

To achieve "Encapsulation" - which is the process of making sure that "sensitive" data is hidden from users. This is done by declaring fields as private.

Properties and Encapsulation

Encapsulation mean is to make sure that "sensitive" data is hidden from users. To achieve this, you must:

- Declare fields/variables as **private**
- Provide **public get** and **set** methods, through **properties**, to access and update the value of a **private** field

Properties

- Private variables can only be accessed within the same class (an outside class has no access to it)
- A property is like a combination of a variable and a method, and it has two methods: a **get** and a **set** method:

```
class Program
{
    2 references
    class Course
    {
        private string nameCourse; // field

        2 references
        public string NameCourse // property
        {
            get { return nameCourse; } // get method
            set { nameCourse = value; } // set method
        }
    }

    0 references
    static void Main(string[] args)
    {
        Course myObj = new Course();
        myObj.NameCourse = ".NET Programming"; // Using properties to access field
        Console.WriteLine(myObj.NameCourse);
    }
}
```

Automatic Properties (Short Hand)

C# also provides a way to use short-hand / automatic properties, where you do not have to define the field for the property, and you only have to write **get;** and **set;** inside the property.

The result is the **same**; the only difference is **less code**.

```
namespace HelloWorld
{
    0 references
    class Program
    {
        2 references
        class Course
        {
            2 references
            public string NameCourse // property
            { get; set; } // GET SET method
        }
        0 references
        static void Main(string[] args)
        {
            Course myObj = new Course();
            myObj.NameCourse = ".NET Programming"; // Using properties to access field
            Console.WriteLine(myObj.NameCourse);
        }
    }
}
```

Microsoft Visual Studio Debug Console: .NET Programming

Inheritance (Derived and Base Class)

In C#, it is possible to inherit fields and methods from one class to another. We group the "inheritance concept" into two categories:

Derived Class (child) - the class that inherits from another class

Base Class (parent) - the class being inherited from

To inherit from a class, use the “ : ” symbol.

Example:

```
1 reference
class Vehicle // base class (parent)
{
    public string brand = "Ford"; // Vehicle field
    1 reference
    public void honk() // Vehicle method
    {
        Console.WriteLine("Tuut, tuut!");
    }
}

2 references
class Car : Vehicle // derived class (child)
{
    public string modelName = "Mustang"; // Car field
}

0 references
class Program
{
    0 references
    static void Main(string[] args)
    {
        // Create a myCar object
        Car myCar = new Car();

        // Call the honk() method (From the Vehicle class) on the myCar object
        myCar.honk();

        // Display the value of the brand field (from the Vehicle class)
        // and the value of the modelName from the Car class
        Console.WriteLine(myCar.brand + " " + myCar.modelName);
    }
}
```

Microsoft Visual Studio Debu

Tuut, tuut!
Ford Mustang

C:\Users\NXC\Desktop\EIU\CSE 443 - DotNet Programming\Example
ocess 24612) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable
le when debugging stops.
Press any key to close this window . . .

Polymorphism and Overriding Methods

Polymorphism means "many forms", and it occurs when we have many classes that are related to each other by inheritance.

```
6 references
class Animal // Base class (parent)
{
    3 references
    public void animalSound()
    {
        Console.WriteLine("The animal makes a sound");
    }
}

1 reference
class Pig : Animal // Derived class (child)
{
    0 references
    public void animalSound()
    {
        Console.WriteLine("The pig says: wee wee");
    }
}

1 reference
class Dog : Animal // Derived class (child)
{
    0 references
    public void animalSound()
    {
        Console.WriteLine("The dog says: bow wow");
    }
}
```

```
0 references
class Program
{
    0 references
    static void Main(string[] args)
    {
        Animal myAnimal = new Animal(); // Create a Animal object
        Animal myPig = new Pig(); // Create a Pig object
        Animal myDog = new Dog(); // Create a Dog object
        myAnimal.animalSound();
        myPig.animalSound();
        myDog.animalSound();
    }
}
```

Microsoft Visual Studio Debug Console

```
The animal makes a sound
The animal makes a sound
The animal makes a sound
```

C# provides an option to override the base class method, by adding the **virtual** keyword to the method inside the base class, and by using the **override** keyword for each derived class methods

```
6 references
class Animal // Base class (parent)
{
    5 references
    public virtual void animalSound()
    {
        Console.WriteLine("The animal makes a sound");
    }
}

1 reference
class Pig : Animal // Derived class (child)
{
    4 references
    public override void animalSound()
    {
        Console.WriteLine("The pig says: wee wee");
    }
}

1 reference
class Dog : Animal // Derived class (child)
{
    4 references
    public override void animalSound()
    {
        Console.WriteLine("The dog says: bow wow");
    }
}
```

```
class Program
{
    0 references
    static void Main(string[] args)
    {
        Animal myAnimal = new Animal(); // Create a Animal object
        Animal myPig = new Pig(); // Create a Pig object
        Animal myDog = new Dog(); // Create a Dog object

        myAnimal.animalSound();
        myPig.animalSound();
        myDog.animalSound();
    }
}
```

Microsoft Visual Studio Debug Console

```
The animal makes a sound
The pig says: wee wee
The dog says: bow wow
```

The **abstract** keyword is used for classes and methods:

Abstract class: is a restricted class that cannot be used to create objects (to access it, it must be inherited from another class).

Abstract method: can only be used in an abstract class, and it does not have a body. The body is provided by the derived class (inherited from).

An abstract class can have both abstract and regular methods:

```
2 references
abstract class AnimalAbstract
{
    0 references
    public abstract void animalSound();
    0 references
    public void sleep()
    {
        Console.WriteLine("Zzz");
    }
}

AnimalAbstract myObj = new AnimalAbstract();
```

class HelloWorld.Sample.AnimalAbstract

CS0144: Cannot create an instance of the abstract type or interface 'Sample.AnimalAbstract'

IDE0090: 'new' expression can be simplified

Show potential fixes (Alt+Enter or Ctrl+.)

Example:

```
// Abstract class
1 reference
abstract class Animal
{
    // Abstract method (does not have a body)
    2 references
    public abstract void animalSound();
    // Regular method
    1 reference
    public void sleep()
    {
        Console.WriteLine("Zzz");
    }
}

// Derived class (inherit from Animal)
2 references
class Pig : Animal
{
    2 references
    public override void animalSound()
    {
        // The body of animalSound() is provided here
        Console.WriteLine("The pig says: wee wee");
    }
}
```

```
class Program
{
    0 references
    static void Main(string[] args)
    {
        Pig myPig = new Pig(); // Create a Pig object
        myPig.animalSound(); // Call the abstract method
        myPig.sleep(); // Call the regular method
    }
}
```



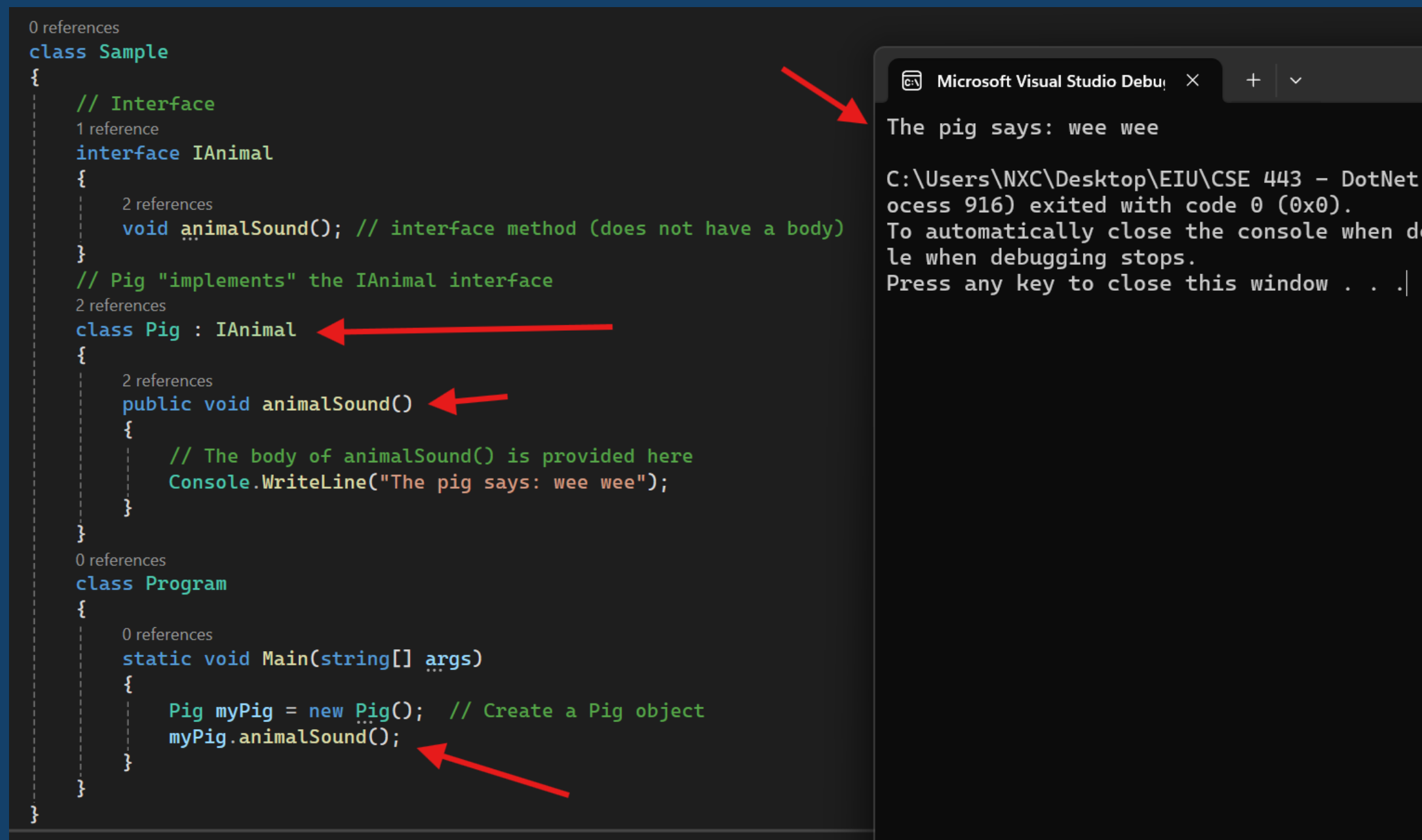
Microsoft Visual Studio Debug Console



```
The pig says: wee wee
Zzz
```

Another way to achieve abstraction in C#, is with interfaces.

An **interface** is a completely "**abstract class**", which can only contain abstract methods and properties (with **empty bodies**):



```
0 references
class Sample
{
    // Interface
    1 reference
    interface IAnimal
    {
        2 references
        void animalSound(); // interface method (does not have a body)
    }
    // Pig "implements" the IAnimal interface
    2 references
    class Pig : IAnimal
    {
        2 references
        public void animalSound()
        {
            // The body of animalSound() is provided here
            Console.WriteLine("The pig says: wee wee");
        }
    }
    0 references
    class Program
    {
        0 references
        static void Main(string[] args)
        {
            Pig myPig = new Pig(); // Create a Pig object
            myPig.animalSound();
        }
    }
}
```

Microsoft Visual Studio Debug Console

The pig says: wee wee

C:\Users\NXC\Desktop\EIU\CSE 443 - DotNet\Process 916) exited with code 0 (0x0).
To automatically close the console when debugging stops, please select an option from the Help menu.
Press any key to close this window . . .

Example in the real project:

```
namespace NXC.Interface;

//this interface to define a common interface
//as a Repository design pattern we'll define a common interface and the other interface will be implement from this
//some common method like a getAll, getById, insert, update, delete ...
71 references | Cường Nguyễn, 62 days ago | 1 author, 1 change
public interface IRepository<T> where T : class
{
    99+ references | Cường Nguyễn, 62 days ago | 1 author, 1 change
    Task<TemplateApi<T>> GetAllAsync(int pageNumber, int pageSize);
    99+ references | Cường Nguyễn, 62 days ago | 1 author, 1 change
    Task<TemplateApi<T>> GetById(Guid id);
    78 references | Cường Nguyễn, 62 days ago | 1 author, 1 change
    Task<TemplateApi<T>> GetAllAvailable(int pageNumber, int pageSize);
    99+ references | Cường Nguyễn, 62 days ago | 1 author, 1 change
    Task<TemplateApi<T>> Update(T model, Guid idUserCurrent, string fullName);
    99+ references | Cường Nguyễn, 62 days ago | 1 author, 1 change
    Task<TemplateApi<T>> Insert(T model, Guid idUserCurrent, string fullName);
    99+ references | Cường Nguyễn, 62 days ago | 1 author, 1 change
    Task<TemplateApi<T>> RemoveByList(List<Guid> ids, Guid idUserCurrent, string fullName);
    78 references | Cường Nguyễn, 62 days ago | 1 author, 1 change
    Task<TemplateApi<T>> HideByList(List<Guid> ids, bool isLock, Guid idUserCurrent, string fullName);
}
```

Example in the real project:

```
namespace NXC.Interface.Interfaces;
```

5 references | Cường Nguyễn, 62 days ago | 1 author, 1 change

```
public interface IEmployeeRepository : IRepository<EmployeeDto>
```

```
{
```

```
    #region ==[ CRUD TABLE Employee ]=====
```

2 references | Cường Nguyễn, 62 days ago | 1 author, 1 change

```
    Task<TemplateApi<EmployeeAndBenefits>> GetEmployeeAndBenefits(Guid idEmployee);
```

2 references | Cường Nguyễn, 62 days ago | 1 author, 1 change

```
    Task<TemplateApi<EmployeeAndAllowance>> GetEmployeeAndAllowance(Guid idEmployee);
```

2 references | Cường Nguyễn, 62 days ago | 1 author, 1 change

```
    Task<TemplateApi<EmployeeDto>> GetEmployeeResigned(int pageNumber, int pageSize);
```

2 references | Cường Nguyễn, 62 days ago | 1 author, 1 change

```
    Task<TemplateApi<EmployeeDto>> FilterEmployee(FilterEmployeeModel model, int pageNumber, int pageSize);
```

2 references | Cường Nguyễn, 62 days ago | 1 author, 1 change

```
    Task<TemplateApi<EmployeeDto>> UpdateEmployeeType(Guid idEmployee, Guid typeOfEmployee,
        Guid idUserCurrent, string fullName);
```

```
    #endregion
```

```
}
```

Start your future at EIU

Thank You